# CineBook: A Movie Ticket Booking System

Final Database Design Document · CS F212 Database Systems · Group 17 · BITS Pilani, Hyderabad Campus · November 2026

Harinandha L (2024A4PS0660H), Imran Hafeez S (2024A4PS0690H), Arjun Rathore (2024A4PS0691H), Sushanth Reddy Kompally (2024A8PS0459H), Pratham Singla (2024A8PS0486H)

---

## 1. Problem statement and scope

Online ticketing platforms must let many users browse shows and reserve seats at once without ever selling the same seat twice. CineBook is a simplified BookMyShow-style system covering cities, venues, screens, movies, shows, seat inventory, bookings, payments and reporting. The project concentrates on relational design and DBMS-level integrity; the application layer only demonstrates it. **In scope:** show search and filtering, seat selection with a temporary hold, simulated payment, cancellation with refund, show scheduling, admin analytics. **Out of scope:** real payment gateways, food ordering, dynamic pricing, seat-map editor.

| User | Main functionality |
|---|---|
| Customer | Register/login; search shows by city, movie, language, date and price; pick seats; apply a coupon; pay; view history; cancel and receive a refund. |
| Venue manager | Schedule shows on screens of the own venue; set per-category seat prices; view own occupancy and revenue. |
| Admin | Maintain cities, venues, screens and movies; dashboards (revenue per venue per month, occupancy per show, top movies per city, peak hours). |

## 2. Assumptions and business rules

R1 A seat is identified only within its screen (weak entity) and has one category: REGULAR, PREMIUM or RECLINER.

R2 A seat has at most one active booking per show; cancelled or expired bookings free the seat but stay as history.

R3 A show is of one movie on one screen; two shows on the same screen may not overlap (back-to-back allowed).

R4 Listed price = SHOW_PRICE of the show and seat category, copied into the reservation as `list_price` (snapshot). `price_paid` exists only after confirmation.

R5 A booking is for one show with 1 to 6 seats. It starts HELD with a 10-minute expiry, becomes CONFIRMED only after successful payment, else CANCELLED or EXPIRED.

R6 Cancellation allowed until 2 h before the show. Assumed refund: 100% if more than 24 h before, 50% between 2 h and 24 h, computed from `price_paid`.

R7 A payment belongs to one booking; a retry creates a new payment record. Payments are simulated.

R8 Every venue has exactly one manager (user with role MANAGER); every user has exactly one role.

R9 Every booking status change is logged in BOOKING_AUDIT.

R10 A booking may use one coupon (percentage or flat, with expiry and optional usage limit); a coupon never makes a booking free. At confirmation the discount is spread so that $\sum$ `price_paid` = `total_amount`.

R11 For fast ticket display, a booking stores a write-once snapshot of show details (movie title, venue, screen, start time), like an invoice.

R12 A booking can only be made for a show that has not started yet.

## 3. ER/EER design

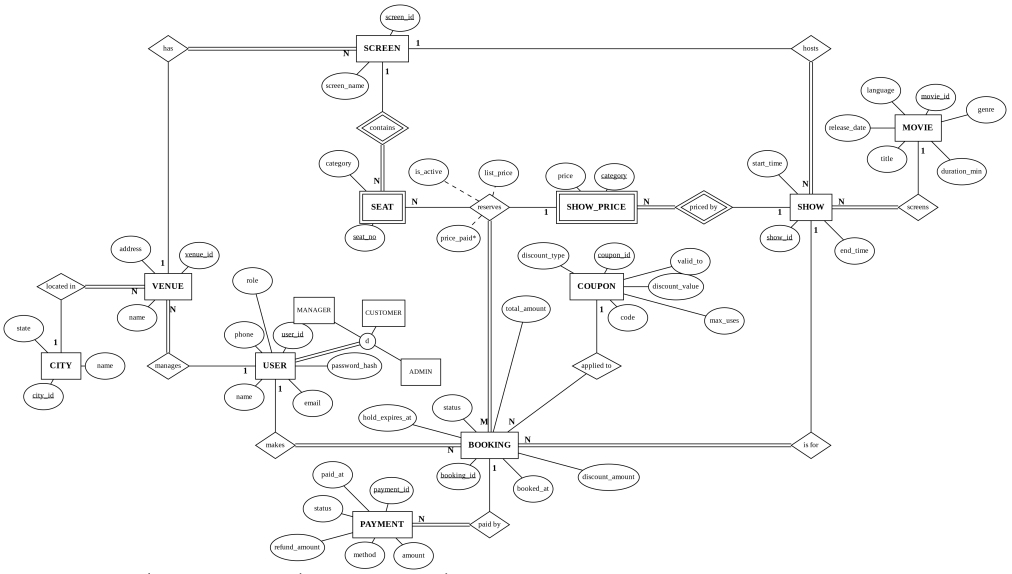



**Figure 1:** Conceptual ER/EER diagram (Chen notation). Foreign keys are not drawn; they appear in the relational mapping.

| Relationship | Entities | Card. | Participation |
|---|---|---|---|
| located in / has | CITY–VENUE; VENUE–SCREEN | 1:N | VENUE total; SCREEN total |
| contains (identifying) | SCREEN, SEAT | 1:N | SEAT total (weak entity, partial key `seat_no`) |
| hosts / screens | SCREEN, MOVIE, SHOW | 1:N each | SHOW total in both |
| priced by (identifying) | SHOW, SHOW_PRICE | 1:N | SHOW_PRICE total (weak entity, partial key `category`) |
| makes / is for | USER, SHOW, BOOKING | 1:N each | BOOKING total in both |
| paid by; applied to | BOOKING–PAYMENT; COUPON–BOOKING | 1:N | PAYMENT total; coupon optional |
| manages | USER (MANAGER), VENUE | 1:N | VENUE total (`VENUE.manager_id`) |
| reserves (ternary) | BOOKING, SEAT, SHOW_PRICE | M:N:1 | each reservation uses one SHOW_PRICE (booking's show, seat's category); attributes `list_price`, `price_paid`, `is_active`; mapped to BOOKING_SEAT |

**EER notes.** USER is specialised into CUSTOMER, MANAGER, ADMIN (disjoint, total). The subclasses have no extra attributes, so it is mapped to one relation with a type attribute `role` (option 8C). SEAT and SHOW_PRICE are weak entities. The ternary *reserves* becomes BOOKING_SEAT, whose composite foreign keys enforce its rules declaratively. Snapshot columns in BOOKING and `seat_category` in BOOKING_SEAT are added during mapping and not drawn (Section 6).

## 4. Relational schema (13 relations)

Timestamps are TIMESTAMPTZ; money is NUMERIC(10,2). Names match `db/01_tables.sql` and `db/02_constraints.sql`.

| Relation | Attributes | Keys and constraints |
|---|---|---|
| APP_USER | (user_id, name, email, phone, password_hash, role) | email UNIQUE NN (plus unique index on lower(email)); role CHECK IN (CUSTOMER, MANAGER, ADMIN) |
| CITY | (city_id, name, state) | UNIQUE (name, state) |
| VENUE | (venue_id, name, address, city_id, manager_id) | city_id → CITY; manager_id → APP_USER (trigger: role = MANAGER); all NN |
| SCREEN | (screen_id, venue_id, screen_name) | venue_id → VENUE ON DELETE CASCADE; UNIQUE (venue_id, screen_name) |
| SEAT | (screen_id, seat_no, category) | screen_id → SCREEN CASCADE; category CHECK IN (REGULAR, PREMIUM, RECLINER); UNIQUE (screen_id, seat_no, category) as FK target |
| MOVIE | (movie_id, title, duration_min, language, genre, release_date) | title, language, duration_min NN; duration_min CHECK $> 0$ |
| MOVIE_SHOW | (show_id, movie_id, screen_id, start_time, end_time) | movie_id → MOVIE; screen_id → SCREEN; CHECK end > start; UNIQUE (show_id, screen_id); EXCLUDE (screen_id =, time range &&) forbids overlap |
| SHOW_PRICE | (show_id, category, price) | show_id → MOVIE_SHOW CASCADE; category CHECK as SEAT; price CHECK $> 0$ |
| COUPON | (coupon_id, code, discount_type, discount_value, valid_to, max_uses) | code UNIQUE NN; type CHECK IN (PERCENT, FLAT); value $> 0$ ($\leq 100$ for PERCENT); max_uses NULL = unlimited |
| BOOKING | (booking_id, user_id, show_id, coupon_id, status, booked_at, hold_expires_at, discount_amount, total_amount, movie_title, venue_name, screen_name, show_start) | FKs to APP_USER, MOVIE_SHOW, COUPON (NULL = none); status CHECK IN (HELD, CONFIRMED, CANCELLED, EXPIRED); discount $\geq 0$; total $> 0$; snapshot columns NN, written once; UNIQUE (booking_id, show_id); index (user_id, booked_at DESC) |
| BOOKING_SEAT (from *reserves*) | (booking_id, screen_id, seat_no, show_id, seat_category, list_price, price_paid, is_active) | four composite FKs (below); list_price $> 0$; price_paid NULL until confirmed, then $0 \leq$ price_paid $\leq$ list_price; partial UNIQUE index (show_id, screen_id, seat_no) WHERE is_active |
| PAYMENT | (payment_id, booking_id, method, amount, status, paid_at, refund_amount) | booking_id → BOOKING NN; method CHECK IN (UPI, CARD, NETBANKING, WALLET); status CHECK IN (INITIATED, SUCCESS, FAILED, REFUNDED); amount $> 0$; refund NULL or $0 \leq$ refund $\leq$ amount |
| BOOKING_AUDIT | (audit_id, booking_id, old_status, new_status, changed_at) | booking_id → BOOKING; filled only by a trigger |

**Why four composite foreign keys in BOOKING_SEAT?** A seat can be booked only if (a) the booking belongs to the same show: $(\text{booking_id}, \text{show_id}) \rightarrow$ BOOKING; (b) the show runs on the seat's screen: $(\text{show_id}, \text{screen_id}) \rightarrow$ MOVIE_SHOW; (c) the seat exists with the stored category: $(\text{screen_id}, \text{seat_no}, \text{seat_category}) \rightarrow$ SEAT; (d) that category has a price for the show: $(\text{show_id}, \text{seat_category}) \rightarrow$ SHOW_PRICE. The DBMS rejects any inconsistent row without a trigger. With the partial unique index and the exclusion constraint, the main rules live in the schema itself.

**Referential actions.** CASCADE only for owned weak entities (SEAT, SHOW_PRICE, SCREEN under VENUE). RESTRICT for history (BOOKING, PAYMENT, BOOKING_AUDIT), so deleting a show, seat or user that has bookings is refused and history is never lost silently.

### 5. Integrity constraints enforced by the DBMS

**Declarative:** a primary key on every relation, a foreign key for every relationship, NOT NULL on mandatory attributes, UNIQUE keys, 18 named CHECK constraints (roles, categories, statuses, payment methods, prices $> 0$, durations, amounts, time order, $0 \leq$ price_paid $\leq$ list_price) and one exclusion constraint.

**No double booking (R2):** the partial unique index `uq_active_seat_per_show` allows one active row per (show, seat); of two concurrent transactions for the same seat, the second fails with a unique violation. **No overlapping shows (R3):** `excl_show_no_overlap` on MOVIE_SHOW forbids overlapping half-open time ranges on the same screen (needs `btree_gist`).

**Rules a CHECK cannot express: 6 triggers.**

| Trigger | Fires | Rule |
|---|---|---|
| `trg_venue_manager_role` | VENUE insert/update | Manager of a venue must be a user with role MANAGER (R8). |
| `trg_booking_audit` | BOOKING insert and every status change | Writes a BOOKING_AUDIT row for the new booking and each status change (R9). |
| `trg_release_seats` | BOOKING becomes CANCELLED/EXPIRED | Sets `is_active=false` on its seats so they can be booked again. |
| `trg_booking_seat_count` (deferred constraint trigger) | BOOKING_SEAT insert/delete, at commit | A booking has 1 to 6 seats (R5); checked at commit so all seats can be inserted first. |
| `trg_booking_confirmation_check` (deferred constraint trigger) | BOOKING insert, status or total change, at commit | A CONFIRMED booking has a SUCCESS payment equal to `total_amount`, every active seat has `price_paid`, and $\sum$ `price_paid` = `total_amount`. |
| `trg_booking_show_in_future` | BOOKING insert | The show must not have started (R12); the seed script bypasses it only via a session setting. |

**Stored procedures and functions.** `sp_create_booking` (expire old holds, lock and validate the coupon row `FORUPDATE`, hold seats, copy `list_price`, compute `discount_amount` and `total_amoun t`); `sp_confirm_booking` (record payment, set `price_paid` per seat, confirm); `sp_cancel_booking` (2-hour cutoff, 100%/50% refund, recorded in PAYMENT); `sp_expire_holds`; `sp_record_failed_p ayment`; `fn_seat_map(show_id)` returns every seat with status and price.

## 6. Normalization and design justification

| Relations | Non-trivial functional dependencies | Normal form |
|---|---|---|
| APP_USER, CITY, VENUE, SCREEN, MOVIE, COUPON, PAYMENT, BOOKING_AUDIT | Every non-key attribute depends on the surrogate key; other candidate keys: email, code, (name, state), (venue_id, screen_name) | BCNF |
| SEAT, SHOW_PRICE, MOVIE_SHOW | (screen_id, seat_no) → category; (show_id, category) → price; show_id → movie_id, screen_id, times; (screen_id, start_time) → show_id | BCNF |
| BOOKING | booking_id → all; show_id → movie_title, venue_name, screen_name, show_start (snapshot) | Not 3NF, by design |
| BOOKING_SEAT | key → rest; booking_id → show_id; show_id → screen_id; (screen_id, seat_no) → seat_category; list_price is a snapshot, so (show_id, seat_category) → list_price is not enforced | Not 2NF, by design |

**Deliberate denormalizations.** (1) *Sale-time snapshots* (`BOOKING.movie_title,venue_name,scr een_name,show_start`; `BOOKING_SEAT.seat_category,list_price`): like an invoice, a ticket must show what was sold even if a movie is renamed or a price changes; they are written once and never updated, so there is no update anomaly. (2) `BOOKING_SEAT.show_id,screen_id` repeat what the booking and show determine; they let the partial unique index and composite FKs enforce R2 in the DBMS, and the FKs make drift impossible. `is_active` mirrors the booking status because a unique index cannot look into another table; a trigger keeps them in step. (3) `total_amount` is derived and maintained only by the booking procedure.

## 7. Key design decisions

- **Rules in the database, orchestration in the app.** Constraints, triggers and procedures own the business rules; the Express back end only opens transactions and passes errors on (`P0001` → HTTP 400, exclusion violation → HTTP 409).
- **Partial unique index** instead of a trigger for "one active booking per seat per show": atomic, race-safe, no locking code.
- **Exclusion constraint with a half-open range** $[\text{start}, \text{end})$: back-to-back shows allowed, overlap impossible even under concurrent inserts.
- **Snapshot read path.** Hot reads (ticket detail, "my bookings") touch only BOOKING and

BOOKING_SEAT, with no joins to user, show, movie, screen or venue; joins and aggregates are kept for admin reports.
- **Weak entities with composite keys** mirror the domain; a single USER table with a role avoids three empty subclass tables.

### 8. Implementation evidence

**Architecture and stack:** browser → React/Vite front end (nginx, port 8080) → Express REST API (port 5050, role-checked JWT routes) → PostgreSQL 16 (port 5432), all started by Docker Compose. React + Vite, Node.js/Express (no ORM, parameterised SQL, JWT, bcrypt), PostgreSQL 16 (`dockercomposeup--build-d`, then `dockercomposerun--rmseed`). **UI overview:** customers use Home, Login/Register, Shows (search and filters), Show details, Seat selection (seat map and coupon), Checkout and My Bookings (view, cancel); managers use the Manager Dashboard and Schedule Show; admins use the Admin Dashboard and the cities, venues, screens and movies pages. React + Vite, Node.js/Express (no ORM, parameterised SQL, JWT, bcrypt), PostgreSQL 16, Docker Compose (`do ckercomposeup--build-d`, then `dockercomposerun--rmseed`). **Data:** 10,044 seeded rows: 51 users, 5 cities, 10 venues, 25 screens, 1,970 seats, 20 movies, 300 shows, 900 show prices, 5 coupons, 988 bookings, 2,728 booking seats, 1,014 payments, 2,028 audit rows.

| # | Query (view) | SQL techniques | Used by |
|---|---|---|---|
| Q1 | `v_revenue_venue_month` | multi-table joins, GROUP BY, `date_trunc`, SUM | Admin dashboard |
| Q2 | `v_show_occupancy` | joins, lateral/correlated aggregate, COUNT FILTER | Admin, manager |
| Q3 | `v_top_movies_city` | CTE, window function ROW_NUMBER() OVER (PARTITION BY city), top 3 | Admin dashboard |
| Q4 | `v_peak_hours` | GROUP BY on EXTRACT(HOUR) with time-zone conversion | Admin dashboard |
| Q5 | `v_show_search` | 5-table join plus three LATERAL aggregates (price range, total seats, seats taken); city, movie, date, price filters | Customer search |
| Q6 | `v_ticket`, `fn_seat_map()` | snapshot read path; seat map with status and price | Ticket, seat selection |

**DBMS features used** (guideline minimum is 3): transactions (booking, payment, cancellation, show scheduling), 6 views, 6 triggers, 5 stored procedures plus 1 function, 12 indexes including partial and functional ones, and database-level constraints throughout.

| # | End-to-end workflow | What it demonstrates |
|---|---|---|
| W1 | Customer: search, seats, coupon, hold, pay, confirm, cancel with refund | One transaction over BOOKING, BOOKING_SEAT, PAYMENT; rollback on failure; seat races resolved by the unique index |
| W2 | Admin dashboard: revenue, occupancy, top movies, peak hours | Complex retrieval with joins, GROUP BY, CTEs, windows and views |
| W3 | Manager schedules a show; invalid operations are attempted | Exclusion constraint, partial unique index, CHECK, triggers; show and its prices inserted in one transaction |

| Verification | Result |
|---|---|
| Constraint tests (`db/test s/w3_tests.sql`) | 38 of 38 invalid operations correctly rejected (overlapping show, double booking, 7 seats, bad price, expired coupon, wrong manager role, broken FKs, and more); all rolled back |
| API integration tests | 30 of 30 passed against a freshly seeded database (register/login, hold, pay, cancel, coupon, role blocking, manager scheduling, overlap rejection) |
| Load tests (300 requests, 20 concurrent, MacBook) | `/shows`: about 470–510 req/s, p95 about 82 ms; `/shows/1/seats`: about 1,500–2,300 req/s, p95 20–33 ms; 0 errors |
| `EXPLAINANALYZE`: snapshot vs naive 8-table join (median of 7 runs, development sandbox) | Ticket detail 0.020 ms vs 0.217 ms execution (about 11×), planning 0.10 ms vs 1.22 ms; my bookings 0.54 ms vs 0.90 ms (about 1.7×). Small data, so absolute times are tiny; the gain is in plan size and planning cost. Script: `db/tests/explain_comparison.sql` |

**Reproducibility.** The repository contains the complete front end, back end, database scripts (`db/0 0`–`db/05`), seed generator, tests, Docker files and a README with setup and run steps, matching the version demonstrated in class.